

Introduction



MARIO PRO2

OBJECT ORIENTED PROGRAMMING BASED PROJECT IN C++
FOR THE COURSE PRO2 AT THE BARCELONA SCHOOL OF INFORMATICS (FIB-UPC)

BY:

Reuben George Jacob (reuben.george.jacob@estudiantat.upc.edu)

Q2 2025 - 2026

Group 52

PROFESSOR IN CHARGE:

Dr. Alejandro Ivan Paz Ortiz (ivan.paz@upc.edu)

INTRODUCTION

As part of the second installment of learning how to program in the language C++ , a fundamental component of the same has been its ability or functionality, to program a project in parts, working with various “Objects”, called Object Oriented Programming(also known by its abbreviation, O.O.P), of which C++ is a well known language for the same.

The advantages of O.O.P:

- We can build programs from standard working modules that are linked to each other instead of cumbersome typing out the code every time.
- Break the program into smaller “modules” that help save time and productivity for the programmer.
- This also brings down exponentially maintenance costs and better software.
- The concept of private elements allows the programmer to secure programs that aren't interfered by other lines of code.
- Scalability approach (from small-scale to large-scale projects).

Disadvantages of O.O.P:

- Sometimes, the length of the program using OOP can be larger in size than the normal/procedural approach. This can cause issues of slower execution of the program.
- It is not suitable for all types of problems/projects.
- Can be tedious to understand for novice programmers to get hold of at the first try.

Objectives:

Through this project, students are being exposed to a practical approach to O.O.P and its principles, especially of classes, objects, encapsulation and implementation. *Concepts such as Inheritance and Polymorphism are **NOT** dealt with in this course or project.*

PRE/CO-REQUISITES:

At the start of the course PRO2 this semester, I was able to pick up on basics of O.O.P through the classes imparted for the same, and ongoing classes about Pointers and the implementation of data structures such as Vectors, Stacks , Queues , Lists and Trees.

DEVELOPMENT PROCESS AND TIMELINE OF THE PROJECT:

The source code for this project is publicly available in a GitHub repository, which was used for version control, documentation preparation, and easy access. The repository can be accessed at:

<https://github.com/reubenlags/PRO2-Practica-Mario>

The following table shows the different stages or steps I've followed to complete the project. This tracker also serves as a monitor of my progress in the skills I've gained, and the mistakes made during the course of this project.

<i>Task/Function:</i>	<i>Date:</i>
<i>Part 0 (Learnt how the dependence between the files, their functionalities; implemented basic methods; learnt game coordinates and the importance of line of code order)</i>	<i>9/5/2026</i>
<i>Part 1</i>	<i>20/5/2026</i>
<i>Part 2</i>	<i>25/5/2026</i>
<i>Part 3</i>	<i>31/5/2026</i>
<i>EXAM/ final supervision</i>	<i>3/6/2026</i>

0 PART 0

PART 0

ABSTRACT:

This part of the project involves studying the given project, especially the source code and the functionality of each class and the objects found within them.

Project Architecture:

1. [utils.hh](#) : useful functions/methods such as painting:
 1. Horizontal lines.
 2. Vertical lines
 3. Sprites.
2. [geometry.hh](#) : declaration of two tuples Pt and Rect.
3. [platform.hh](#): involves the platform found in the game.
4. [mario.hh](#): the Mario sprite and relevant physics for the same.
5. [game.hh](#) : the game mechanics and aesthetics
6. [main.cc](#) : the main loop of the game and principal game.

The suggested solutions to the following questions are as follows (see from next page):

PART 0

Exercise 0.1: Fes els canvis necessaris al codi del projecte que permeti pausar el joc amb la tecla 'P', i tornar-lo a engegar tornant a prémer 'P'. Pausar el joc vol dir que l'acció queda congelada en el mateix fotograma permanentment, fins que no es torni a posar en marxa. Això implica que no s'actualitzen els objectes ni la càmera, però sí que se segueix pintant el joc (i es processen les tecles!). Els canvis en aquest exercici són: a) afegir un camp nou a la classe Game , i modificar un o més mètodes existents.

1. Creation of a new camp/field in [game.hh](#) under the name “paused_“ of type bool.

```
1. bool paused_;
```

2. To detect whether the 'P' key was pressed, we must first create a conditional under the process_keys method in the [game.cc](#) file. This acts like a toggle detector for this key.

```
3. void Game::process_keys(pro2::Window& window) {  
4.     else if (window.was_key_pressed('P')) {  
5.         paused_ = !paused_;  
6.     }  
7. }
```

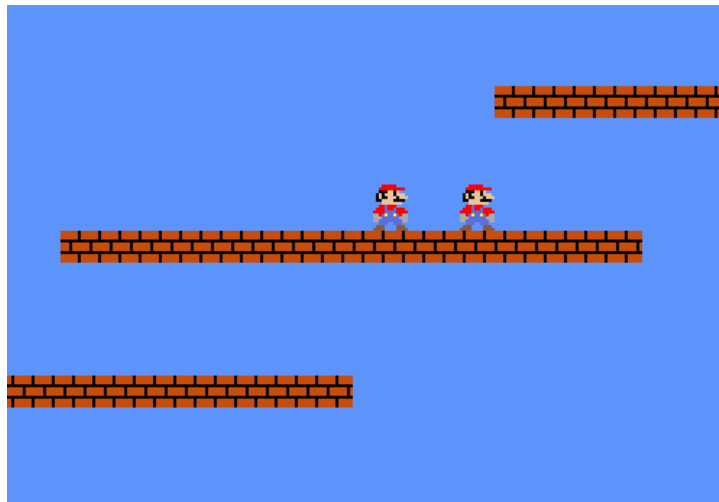
3. Once the key has been pressed again to resume the game, we must be able to update the objects and the camera of the window, (keeping in mind that the key must always be processed!)

```
void Game::update(pro2::Window& window) {  
    if (not paused_)  
        update_objects(window);  
        update_camera(window);  
}
```

PART 0

Exercise 0.2: Fes els canvis necessaris (de nou afegir membres i canviar mètodes) per tenir un altre Mario al joc. Abans de fer-ho, pensa bé si es tracta d'una nova classe o d'un nou objecte! Posa'l com a membre de la classe Game amb nom `mario2_`. Inicialitza'l amb una coordenada x uns 30 píxels a l'esquerra del `mario_` principal. Busca els llocs a on apareix el primer Mario i fes les mateixes crides als mateixos mètodes que els que es fan amb `mario_`. Abans de provar-ho, què penses que succeirà quan executis? Els dos Marios són independents? Ja tenim un joc per a dos jugadors?

I was able to create a duplicate sprite (`mario2_`), declaring it as an object of type 'Mario' and creating relevant changes in existing methods. However, the two objects don't act independently of one another, instead moving simultaneously when the arrow keys are used. Hence, the possibility of a two-player mode won't be feasible at this stage.



PART 0

Exercici 0.3 (dificultat mitjana): Fes que mario2_ es pugui controlar amb tecles diferents que mario_ . Caldria primer modificar la classe Mario de la següent manera. Primer cal que Mario pugui guardar les 3 tecles que el controlen en camps privats, que es poden dir jump_key_ , left_key_ , i right_key_ . En el projecte inicial, les tecles són fixes i són l'espai i les fletxes d'esquerra i dreta. Pel mario2_ cal que siguin "W" per saltar, "A" per l'esquerra i "D" per la dreta (això permet jugar a dues persones). Fixa't que pots passar un caràcter als mètodes. `Windows::is_key_pressed` i `Window::was_key_pressed` perquè es fan servir els codis ASCII. Els valors d'aquestes 3 tecles s'haurien de passar al constructor de Mario i allà guardar-les en els 3 camps privats. Després cal anar al codi que comprova les tecles que s'han premut i treure els valors fixos i posar-hi els camps privats, que tenen les tecles passades en el constructor. Finalment, en el constructor de Game , caldria construir mario_ amb unes tecles i mario2_ amb unes altres. Fet això dues persones poden jugar al joc. Un moment... i la càmera?? Què passa amb la càmera??

Instead of individually creating conditional statements within the update method, creating parameters for the control keys within the Mario sprite allows us better customization, especially if we have two sprites and they need to be controlled using different sets of keys, as is in this case.

1. Creation of 3 private fields in [mario.hh](#) :

```
int jump_key_;
int left_key_;
int right_key_;
```

2. Adding more parameters to the Mario constructor method :

```
Mario(pro2::Pt pos, int jump_key, int left_key, int right_key) :
pos_(pos), last_pos_(pos), jump_key_(jump_key), right_key_(right_key),
left_key_(left_key) {}
```

3. Update the Mario parameters in [game.cc](#) file:

```
mario_({width / 2, 150}, Keys::Space , Keys::Left, Keys::Right),
mario2_({width / 2 - 30, 150}, 'W', 'A', 'D'), // 0.2, 0.3
```

PART 0

4. Accordingly, changing the parameters from `Keys::Space` to `jump_key_` and so on (mario.cc):

```
5. if (window.is_key_down(jump_key_)) {  
6.     jump();  
7. }  
if (window.is_key_down(left_key_)) {  
    speed_.x = -4;  
} else if (window.is_key_down(right_key_)) {  
    speed_.x = 4;  
}  
if (speed_.x != 0) {  
    looking_left_ = speed_.x < 0;  
}
```

With this following changes, we are able to operate both the sprites using two different set of keys, essentially enabling two-player mode in our game.

PART 0

Exercici 0.4: Simplifica el codi de `Game::update_camera` per tal que la càmera sempre tingui al Mario al centre de la pantalla. Es tracta de demanar la posició del Mario , la posició central de la càmera, i cridar `Window::move_camera` indicant la distància entre la càmera i el Mario . Executa el joc i prova a saltar a les plataformes. Et sembla que la càmera té un moviment bo per jugar?

The previous code made sure that the camera obeyed directional bounds in such a way that there was smooth movement with respect to the sprite. This exercise essentially makes the camera focus, or move with respect to the sprite only which creates a sort of jittery animation and not a user friendly experience to play with, especially considering if the game were to be played in multiplayer mode!

The following changes were made to the `Game::update_camera` method in the [game.cc](#) file:

```
void Game::update_camera(pro2::Window& window) {
    const Pt pos = mario_.pos();
    const Pt cam = window.camera_center();

    //0.4
    int dx = pos.x - cam.x;
    int dy = pos.y - cam.y;

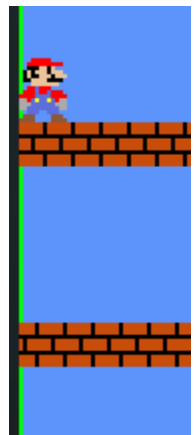
    window.move_camera({dx, dy});
}
```

Instead of the earlier bound based setting of the coordinates of the camera movement, this allows for a complete focus on the sprite movement.

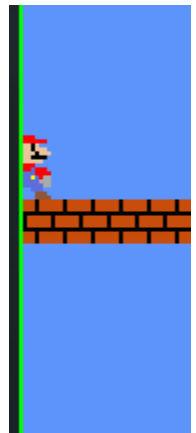
PART 0

Exercici 0.5: En aquest exercici farem servir les utilitats del mòdul `utils.hh` anomenades `paint_hline` i `paint_vline` que pinten una línia horitzontal i una línia vertical, respectivament. Es tracta de d'afegir codi a `Game::paint` per dibuixar un requadre al marc de la pantalla, però en comptes de posar coordenades fixes i que el requadre sembli que està en el món del Mario, cal que el requadre es mogui amb la càmera i sembli que realment és un requadre que es mou amb la pantalla. Pinta el requadre l'últim de tots o bé just després de `window.clear(...)` per veure la diferència. (En aquest exercici hauràs de buscar-te tu mateix els mètodes que et permetin fer el que es demana!)

To be able to create a rectangular frame that fits in the shape of the camera and moves with it as well, creating a rectangle with a `window.camera_rect()` creates our border frame in the shape of the camera of the window. Setting the colour as green and the lines of code, **just after the `window.clear()` line**, as seen in the figure below, a green outline is painted on the border of the game. **But noticeably, the sprite and platform is painted over it and the border has become less visible.**



To test the hypothesis of the importance of the order of lines of code, when the same lines of code **were placed at the end (i.e after painting the sprites and the platforms)**, the border line is printed **on top of all the game elements, making the border frame more visible.** This highlights the importance of draw order, especially since UI elements like a border should always be drawn/painted last so game objects don't cover them.



```
//0.5
Rect camera = window.camera_rect();
paint_hline(window, camera.left, camera.right, camera.top, pro2::green );
paint_hline(window, camera.left, camera.right, camera.bottom - 1,
pro2::green );
paint_vline(window, camera.left, camera.top, camera.bottom,
pro2::green);
paint_vline(window, camera.right - 1, camera.top, camera.bottom,
pro2::green);
```

PART 0

Exercici 0.6: Afegeix una funció a util.cc que es digui paint_rect , que rebí un rectangle (de tipus Rect) i un color i pinti tots els píxels del rectangle del color (mira primer paint_hline o paint_vline com a inspiració). Recorda declarar la funció a utils.hh !

First, lets declare the method in [utils.hh](#) file:

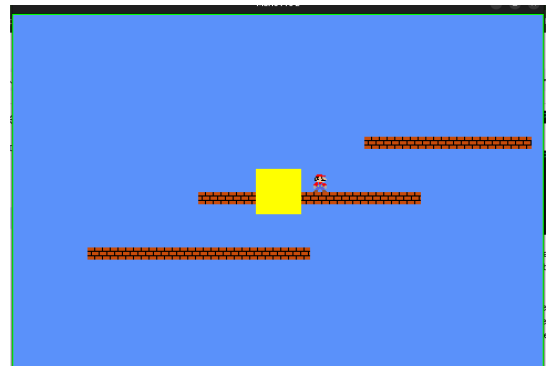
```
void paint_rect(pro2::Window& window, Rect r, Color color);
```

Then we implement this method in [utils.cc](#) file:

```
void paint_rect(pro2::Window& window, Rect r, Color color){
    for (int i = r.left; i <= r.right; i++){
        for (int j = r.bottom; j <= r.top; j++){
            window.set_pixel({i,j}, color);
        }
    }
}
```

Exercici 0.7: Utilitzant paint_rect , pinta un rectangle de color groc al mig de la pantalla a sobre de tots els altres objectes. Prova també de posar el rectangle al fons de tot, per sota inclús de les plataformes.

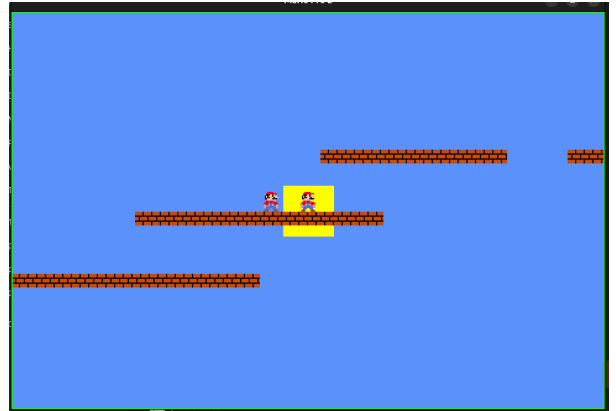
To create a rectangle that has its coordinates at the center of the screen, in game.cc , we first declared a rectangle that has the central coordinates of the screen and called the paint_rec function to paint the rectangle. Since it was called AFTER the paint sprites functions were written, the rectangle overlaying over all the other elements such as the sprite and the platform.



```
//0.7
Pt centre = window.camera_center();
Rect r = {centre.x - 20, centre.y - 20, centre.x + 20, centre.y + 20};
paint_rect(window, r, pro2::yellow);
```

PART 0

Writing the same lines of code write before the platforms and mario sprites are painted makes the rectangle painted underneath all the game objects (although, just above the blue sky!). This further emphasizes the role of the order of code writing in this project.



Exercici 0.8: Com faries perquè el rectangle groc fes pampallugues, és a dir, que aparegués i desaparegués ràpidament en seqüència?

To implement a functionality that involves being able to “blink” the rectangle constantly may require consulting of the frames per second (fps) and allowing the rectangle to paint every alternate frame. To do so, we can create an if conditional in the game::paint method and recreate the same steps followed to make the rectangle within it.

```
if (window.frame_count() % 75 < 25) {  
    Pt centre = window.camera_center();  
    Rect r = {centre.x - 20, centre.y - 20, centre.x + 20, centre.y +  
20};  
    paint_rect(window, r, pro2::yellow);  
}
```

The blink logic is as follows;

- Frames 0-24 → **visible** (25 frames $\approx$ 0.5 seconds)
- Frames 25-74 → **invisible** (50 frames $\approx$ 1 second)

Which keeps repeating.

0 PART 1

PART 1

ABSTRACT:

This part of the project involves creating a new object that the Mario sprite can collect (i.e similar to that of the coins that is found in the original game).

To start, I have decided to create an object with a **cross shape** and as per the requirements, the cross must:

- Exist in numerous amounts (similar to the spaced out coins in the original game).
- Have a position on the screen.
- Be collectible by the Mario sprite.
- Disappear when collected.
- Have a simple animation (i.e floating up and down).

Since I have to create a new object, just like how the mario sprite was encapsulated and implemented, I'll be creating a [cross.hh](#) and [cross.cc](#) file.

In the [cross.hh](#) file this was the initial line of code:

```
#ifndef CROSS_HH
#define CROSS_HH

#include <iostream>
#include <vector>
#include "geometry.hh"
#include "utils.hh"
#include "window.hh"

using namespace pro2;

class Cross{
    private:
        Pt pos_; // position of the cross
        bool collected_; // check if the cross was collected
        int frame_count_; // create animation
```

PART 1

```
public:
    Cross(Pt pos) : pos_(pos), collected_(false) , frame_count(0) {}
    Pt pos() const { return pos_;}
    void paint(Window& window){}
    void update();
    bool is_collected() const { return collected_;}
    void collect() { collected_ = true; }

};

#endif
```

Project tracker

Last updated May 30, 2026

Project tracker

Tasks

Assign tasks for each project

Part 0

Task	Status	Due
Reading and deep analysis of the documentation	In progress ▾	📅 D
Exercise 0.1	Completed ▾	📅 D
Exercise 0.2	Completed ▾	📅 D
Exercise 0.3	Completed ▾	📅 D
Exercise 0.4	Completed ▾	📅 D
Exercise 0.5	Completed ▾	📅 D
Exercise 0.6	Completed ▾	📅 D
Exercise 0.7	Completed ▾	📅 D
Exercise 0.8	Completed ▾	📅 D

Part 1

Task	Status	Owner
Add task	Completed ▾	 P
Add task	Not started ▾	 P

Project 3

Task	Status	Owner
Add task	Blocked ▾	 P
Add task	In progress ▾	 P